

```
print("address before the change: ", id(tuple1))
tuple1 = (1, 2, 3, 4)
print(type(tuple1))
print(tuple1)
print("address after the change: ", id(tuple1))
The output is:
```

```
address before the change: 140509666846336
<class 'tuple'>
(1, 2, 3, 4)
address after the change: 140509667383792
```

Changing the list inside tuple

We know that the tuple can hold any value. We have tried to save a list inside tuple. Let's try editing its value. Can we edit? Will it change the list? Let's find out:

```
t = (1, 2, [10, 20, 30])
print(id(t), t)
t[2] += [40, 50]
```

The output is:

```
140509666702784 (1, 2, [10, 20, 30])
```

```
TypeError                                 Traceback (most
recent call last)
<ipython-input-15-cc42d1fec8cf> in <module>
      1 t = (1, 2, [10, 20, 30])
      2 print(id(t), t)
----> 3 t[2] += [40, 50]
```

```
TypeError: 'tuple' object does not support item assignment
```

It has clearly thrown an error, so it should not have updated the values as well:

```
print(id(t), t)
```

The output is:

```
140509666702784 (1, 2, [10, 20, 30, 40, 50])
```

But if you see carefully, the values are appended. This is an edge case where Python behaves strangely. So, putting mutable items in tuples is not a good idea.

Memory allocation

Check the memory allocated – a tuple uses only required memory. It is not overallocated as it is not resizable:

```
print(sys.getsizeof(tuple1))
```

The output is:

```
72
```

Reuse memory

To reduce memory fragmentation and speed up allocations, Python reuses old tuples. If a tuple is no longer needed and has less than 20 items, instead of deleting it permanently, Python moves it to a free list and uses it later.



Note: Though it's not certain always, the same memory will be used.

```
a = (1,2)
print(id(a))
del a
b = (1,2)
print(id(b))
```

The output is:

```
140509665739520
140509665739520
```

Empty tuple

When two empty tuples are created, they will point to the same address space. Empty tuples act as singletons, that is, there is always only one tuple with a length of zero. When creating an empty tuple, Python points to the already preallocated one in such a way that any empty tuple has the same address in the memory. This is possible because tuples are immutable, and sometimes this saves a lot of memory:

```
a = ()
b = ()
if a is b :
    print("A and B are not mapped to same address")
    print("address of a: ", id(a))
    print("address of b: ", id(b))
```

```
print("A size of list: ", sys.getsizeof(a))
print("B size of list: ", sys.getsizeof(b))
```

The output is:

```
A and B are not mapped to same address
address of a: 140509600395328
address of b: 140509600395328
A size of list: 40
B size of list: 40
```